

UFR de mathématique et d'informatique

Université de Strasbourg



University of Strasbourg

Department of Computer Science

Optimizing Code Generation for Embedded Systems with Real-Time Operating Systems

Jaenai RUGENGANDE IHIMBAZWE

Supervisor: Pierre David

A dissertation submitted in fulfilment of the requirements of
the Department of Computer Science for the completion of the TER project

May 4, 2025

Declaration

I, Jaenai RUGENGANDE IHIMBAZWE, of the Department of Computer Science, University of Strasbourg, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this dissertation are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my dissertation being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of University of Strasbourg and public with interest in teaching, learning and research.

Jaenai RUGENGANDE IHIMBAZWE

May 4, 2025

Abstract

Real-time systems are an important part of modern technology such as automotive systems, aviation, telecommunication, and industrial automation. The main characteristic of these systems is the need to execute tasks within a more or less strict time limit, where missing a deadline can lead to severe consequences.

In this dissertation, I provide an overview of the challenges and solutions so far in achieving both reliability and efficiency of code run in real-time environments. By reviewing existing research, I present a survey of scheduling techniques and dive into methods of optimizing code generation tailored to real-time operating systems(RTOS). I focus on strategies that not only meet real-time constraints but also enhance performance, ultimately contributing to the development of more robust and responsive embedded applications.

Acknowledgments

First and foremost, I would like to thank my project supervisor, Pierre David, for his support and guidance throughout the project. I am grateful to him for the time he invested in assisting me with my work and for his advice and expertise. His support was important in keeping me on the right path, and I would not have been able to do as much without him.

I would also like to thank Basile Sauvage, Pascal Mérindol, and Fabrice Théolyre for their TER-related courses. They played an important role in solidifying my understanding of research methodology as well as providing me with essential tools such as how to read and summarize research articles properly and how to write a related work section.

Contents

List of Figures	v
List of Tables	vi
1 Introduction	1
2 Real-Time Operating Systems	2
2.1 Different levels of real-time constraints of systems	2
2.2 Characteristics of a task	3
2.3 Different types of tasks	3
2.4 Solutions proposed by RTOSes	4
3 Scheduling	8
3.1 Different types of scheduling	8
3.1.1 Rate Monotonic Priority Assignment(RM)	8
3.1.2 Earliest Deadline First Scheduling (EDF)	10
3.1.3 Servers for aperiodic tasks	10
3.1.4 Why RM and EDF?	11
3.2 Fault-tolerance	11
3.3 Architecture-awareness	11
4 Optimizing code generation	14
4.1 The Need for WCET-Aware Compilation	14
4.1.1 The Role of Control Flow Information	15
4.1.2 Managing Control Flow Information During Optimization	15
4.1.3 WCET-Aware Compiler Design and Techniques	16
4.2 Multiobjective optimization	17
4.3 Hardware-awareness	19
5 Synthesis of Concepts and Applications: A Tabular Overview	21
6 Conclusion and reflection	22
References	23
Appendices	27
A Mathematical Foundations of Pareto Optimality	27
B The Strength Pareto Evolutionary Algorithm 2 (SPEA2)	28
C Clustering solutions	29

List of Figures

2.1	Representation of a real-time task (Extracted from [1])	3
2.2	Example of a scenario tested in FreeRTOS for ensuring task management (Extracted from [2])	5
2.3	Example of intertask communication (Extracted from [3])	6
2.4	Memory management in VRTX (Extracted from [4])	7
3.1	Example of RM on multiple instances of τ_1 and τ_2 (Extracted from [1]) .	9
3.2	Example of EDF on multiple instances of τ_1 and τ_2 (Extracted from [1]) .	10

List of Tables

4.1	Comparison of Single-Core and Multi-Core Compilation Based on [5] . . .	20
5.1	A synthesis of the real-time concepts applied in three RTOSes	21

List of Abbreviations

OS	Operating System
RTOS	Real-Time Operating System
WCET	Worst-Case Execution Time
ACET	Average-Case Execution Time
MPSoC	MultiProcessor System on Chip
PL	Programmable Logic
WCC	WCET-Aware C Compiler
RM	Rate Monotonic
EDF	Earliest Deadline First
SPM	Scratch-Pad Memory
DMA	Direct Memory Access
ISR	Interrupt Service Routine
EMO	Evolutionary Multi-Objective
IPC	Inter-Process Communication
SPEA2	Strength Pareto Evolutionary Algorithm 2
CFG	Control-Flow Graph
BRAM	Block RAM

1 Introduction

Having worked with real-time systems during one of my previous internships, I became curious about their role in modern technology. I was fascinated by how they work and how it is possible to implement applications under their required timing constraints. It seemed very difficult to ensure coded tasks follow a precise order and time limit.

Through my compilation class this year, I gained basic knowledge of how code is compiled to run on a system. From lexical analysis to low-level code generation, there are many transformations that are done. Often, these transformations optimize the code for efficiency. One example is loop-invariant code motion, which involves moving a calculation that always gives the same result outside of a loop.

The more I explored these two domains, real-time systems and compiler optimizations, the more I started to wonder how they interact. That curiosity led me to propose a research subject exploring how the code generation process can be optimized for embedded systems with real-time constraints. Specifically, I began to question whether traditional performance-driven compiler optimizations might, in fact, conflict with the need for strict timing guarantees in such systems.

What if the very optimizations that make general-purpose code more efficient could compromise the reliability of real-time systems? In applications where milliseconds can mean the difference between success and failure (e.g. automotive, aerospace, or medical applications) code behavior is not just about performance; it is about predictability and guarantees.

Exploring this question opens up a rich intersection between compiler theory and system design. A familiarity with the distinctions between real-time operating systems (RTOS) and their non-real-time counterparts, especially in terms of scheduling approaches and system guarantees, is essential. Furthermore, this knowledge clarifies why certain optimizations that are valuable for ordinary software must be re-examined or entirely excluded in applications where deadlines are absolute.

For anyone interested in low-level systems, performance-critical applications, or even embedded development, this topic offers a meaningful glimpse into how software must adapt when timing is not just a benchmark, it is a constraint. While the development of a full framework remains a longer-term objective, the conceptual groundwork laid here highlights critical intersections between compilation strategies and the temporal guarantees required by real-time operating systems. It offers a foundation for further research into how compiler optimizations can be adapted, or redefined to align with the stringent demands of real-time constraints.

2 Real-Time Operating Systems

An operating system (OS) is a software that manages hardware and system resources for running programs. It ensures isolation between programs and handles their scheduling. The OS creates the illusion that each program is running independently, without compromising the execution of others. It provides controlled access to allocated resources at a given time. In general-purpose systems, an OS aims to be fair. It does not need to guarantee strict timing. Delays are considered acceptable because the tasks executing late or early do not lead to critical failures.

However, real-time systems require timing constraints. For example, a pacemaker regulates the conduction system of the heart by monitoring the heart's activity, and generating electrical pulses delivered to one or more chambers of the heart for it to pump blood if needed. These pulses must be generated at precise intervals depending on the noticed abnormalities and any deviation might lead to irreversible consequences. Thus, the tasks of a program run in such a system have a deadline. To address these requirements, the general-purpose operating system must be adapted to accommodate real-time constraints. This is where RTOSes come into play. They provide needed solutions to problems caused by time-critical execution.

In this chapter, we will explore the various levels of timing constraints in real-time systems. All system tasks do not have the same severity when missing a deadline. We will then explore the different types of real-time tasks, distinguishing between periodic and aperiodic tasks, and finally, we will look into the constraints imposed by real-time requirements and the mechanisms RTOSes use to meet them.

2.1 Different levels of real-time constraints of systems

A pacemaker, a robotic assembly line, and a live audio-video system are all real-time systems but they are obviously different in terms of priority. A delay of execution in each of the systems results in different types of consequences, more or less important. A delay of a task in a pacemaker means the failure of the whole system and subsequently a major problem for the user. The failure of a task on a robotic assembly line might lead to the delay of the whole production or even the loss of some products but it can be tolerated and easily repaired without as important of a subsequent problem as for the pacemaker. The failure of a task on a live audio-video system might cause some glitches in the system but nothing important.

As we can see, the classification of real-time systems depends on the gravity of consequences due to failure of a task in the system. It does not depend on how many times it is likely to fail and not how long it takes for a failure to be detected and changed.

We can infer from the given example that there are three types of real-time systems, as also mentioned in [1]:

- Hard Real-Time Systems: if a deadline is missed, there are consequences on the

environment of the system. In the example of the pacemaker, missed deadline could potentially lead to cardiac arrest for the user.

- Firm Real-Time Systems: if a deadline is missed, the results of the system become useless. In the example of a robotic assembly line, let us imagine the process is food packaging and the task of dropping food in can is missed, we would have an empty closed can which would be useless since there is no food inside.
- Soft Real-Time Systems: Deadline is desirable but missing it causes no important problem. In the example of live audio-video system, we would like to have transmissions with no glitches but a couple of glitches will not be problematic.

2.2 Characteristics of a task

A task i executed in real-time systems has some parameters/characteristic figures as illustrated in Figure 2.1 below :

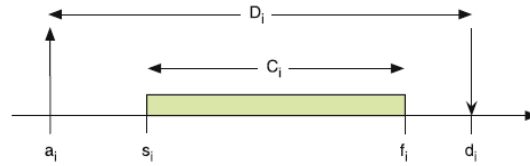


Figure 2.1: Representation of a real-time task (Extracted from [1])

- arrival time (a_i): when it is ready to be executed
- worst-case execution time (C_i) : the time it would take for it to execute in the worst possible scenario
- deadline (D_i/d_i): A deadline can be absolute (d_i) as in the latest time by which a task has to be done or relative (D_i) as in the time between the arrival time and the latest time a task has to be done
- starting time (s_i): when it actually starts to execute
- finishing time (f_i): when it actually finishes to execute

Those characteristics allow RTOS schedulers to do their job according to them. We will see it in more details in the following chapter (3).

2.3 Different types of tasks

Even though we have time characteristics and timing constraints, all the tasks of a real-time system are not executed regularly. There are two types of tasks as mentioned in [1] and [6]:

- **Periodic tasks:** They have a known arrival time and execute each given period. As shown in [1], they are usually on a basis of “sense-execute-act” loop instructions that form the backbone of a system. They help in control structures. In our pacemaker example, it would be the monitoring task. At each given interval, there is a check to see if the heart activity is not irregular.
- **Aperiodic tasks:** They do not have a known arrival time and so their scheduling is a little harder. Examples of such tasks are parameter setting or interrupt handling. Some RTOSes that are event-driven also use such tasks. In [7], there is an example of the event-driven approach in TinyOS where the system completely relies on commands that will start a task.

2.4 Solutions proposed by RTOSes

RTOSes implement specific solutions across core operating system functions to meet the stringent timing requirements of real-time applications, fundamentally differentiating them from general-purpose OSes which prioritize throughput and fairness. These solutions are designed to ensure predictability, determinism, and timely task completion.

- **Predictability and Determinism:**

A cornerstone of RTOS design is the assurance of predictability and determinism. Predictability allows system designers to analyze and verify that all critical tasks will meet their deadlines under all anticipated conditions. Determinism implies consistent and bounded response times for system operations, minimizing unpredictable delays. This contrasts with general-purpose OSes, where such timing may be subject to variable delays due to complex internal operations and resource management strategies not optimized for strict time bounds.

A study done in [8] meticulously analyzed the execution path non-determinism within the Linux kernel, focusing on system calls related to file operations. The research demonstrated that "the execution path of a system call is effectively non-deterministic in Linux kernel space". To investigate this, the researchers executed a simple application that reads from one file and writes to another on a Raspberry Pi 3B running a Linux kernel. They traced kernel-space execution paths using `Ftrace` under various scenarios, including differing system loads and file system types. For instance, when varying system load using the `hackbench` benchmark, the number of unique execution paths for system calls increased significantly with the load average, reaching hundreds of unique paths. This variability in execution paths, even for a simple application with identical inputs, clearly illustrates the inherent non-determinism and lack of predictability in a standard Linux OS, as the precise sequence of kernel functions executed can change significantly based on the system’s dynamic state.

- **Task Management and Scheduling:**

A critical function of an RTOS is the sophisticated management and scheduling of tasks to guarantee deadline adherence. RTOSes employ real-time scheduling algorithms, such as fixed-priority or dynamic-priority scheduling, coupled with pre-emption, to ensure the most critical tasks gain timely access to the CPU. The

schedulability of a system is often verified beforehand to confirm that all critical tasks can meet their deadlines under worst-case conditions as shown in [3]. We will be seeing it in more details in chapter 3.

An illustrative example of RTOS task management and scheduling is the scenario tested in FreeRTOS, as presented in Figure 2.2. This figure depicts a timeline illustrating the scheduling behavior of tasks with different priority levels (T_1 , T_2 , T_3 and T_4) under a priority-based preemptive scheduler that also supports round-robin scheduling for tasks of the same priority. The scenario highlights how a higher-priority thread (T_4 created at t_5) can immediately preempt a lower-priority thread (T_1) to ensure its timely execution. This deterministic response to priority and readiness is a key feature of RTOS scheduling.

In contrast to real-time operating systems designed for temporal guarantees, general-purpose OS schedulers, like the one in Linux, prioritize overall throughput, fairness, and resource utilization, which inherently leads to non-deterministic behavior for specific tasks. The study in [8] provides clear evidence of this lack of time guarantees. Their experiments showed that "system load affects the variability of system calls' execution paths," and importantly, "the effect of variability differs based on the system call and the executed application".

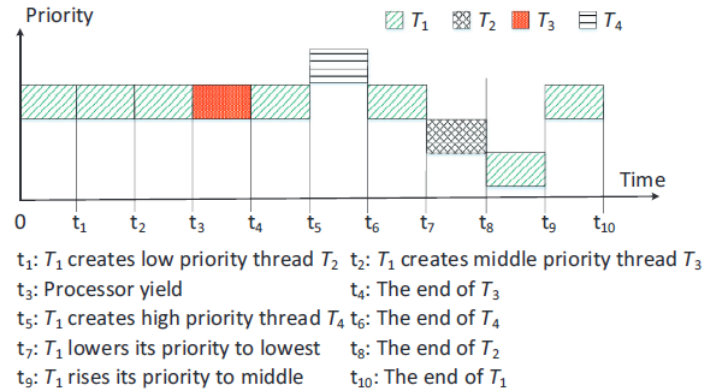


Figure 2.2: Example of a scenario tested in FreeRTOS for ensuring task management (Extracted from [2])

- Inter-Task Communication and Synchronization:

Communication and synchronization between tasks in an RTOS are designed to be predictable, avoiding unbounded delays and priority inversion, which occurs when a high-priority task is blocked by a lower-priority task. RTOSes utilize specialized protocols and communication objects for this purpose.

Figure 2.3 provides a conceptual example of intertask communication in an industrial RTOS. The scheme illustrates tasks (Task1 and Task2) interacting through operations like reserve, require, receive, and release. These represent controlled access to shared resources or data exchange points. When implemented with real-time synchronization protocols (such as priority inheritance or priority ceiling), these mechanisms ensure that tasks waiting for resources are handled in a priority-aware manner, bounding the maximum blocking time and maintaining predictability in task interactions.

Conversely, general-purpose OSe provide various Inter-Process Communication

(IPC) and synchronization primitives, but their behavior might not be strictly priority-aware or provide bounded delays, which can lead to unpredictable performance in real-time scenarios. In [9], the evaluation of IPCs in general-purpose OS shows that they might be affected by factors such as process scheduling, which we have seen is not priority-aware in general OS. This makes it unpredictable.

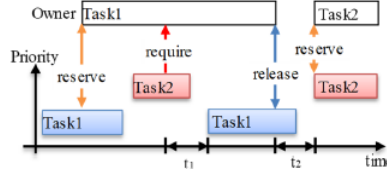


Figure 2.3: Example of intertask communication (Extracted from [3])

- Interrupt Handling:

RTOSes manage interrupts to minimize non-deterministic interference with scheduled tasks. While some interrupts need rapid service, RTOSes integrate interrupt handling with scheduling or employ techniques like disabling interrupts for controlled periods or using polling to ensure that critical tasks are not unduly delayed. An Interrupt Service Routine (ISR) is a special routine executed in response to an interrupt signal. The goal of RTOS interrupt handling is to ensure predictable and low-latency responses to external events.

The MARS operating system described in [10] explicitly disabled most interrupts to the CPU except for the periodic clock interrupt. Interaction with peripheral devices was then realized by polling within the clock interrupt handler. This approach was taken because standard interrupt mechanisms in general-purpose OSes can lead to an unpredictable CPU load and missed deadlines. By controlling interrupt processing in this manner, MARS aimed to maintain determinism.

General-purpose OSes prioritize quickly servicing interrupts, often running ISRs at a very high priority level that can preempt most other tasks. While this ensures responsiveness to the interrupt event itself, the execution time of the ISR and any subsequent processing can introduce unpredictable delays for lower-priority tasks.

- Memory Management:

RTOSes prioritize predictable memory access and allocation times. They often avoid virtual memory and dynamic allocation methods with variable-sized blocks on a heap, which can introduce unpredictable delays due to fragmentation or page faults. Instead, static or fixed-size allocation within partitions is common to ensure deterministic memory access times.

Figure 2.4, illustrating memory management in VRTX, demonstrates a partitioned memory pool. This figure clearly shows distinct memory partitions, each containing blocks of a fixed size (e.g., “Partition 0: 512-byte blocks,” “Partition 1: 100-byte blocks,” “Partition 2: 250-byte blocks”). This partitioning allows the RTOS to manage memory by allocating and releasing blocks from these predefined pools. The use of fixed-size blocks within partitions is a static allocation strategy that ensures deterministic allocation times because the system does not need to search for variable-sized contiguous blocks or manage external fragmentation. This approach

prioritizes predictable timing over memory utilization efficiency and is characteristic of RTOSes.

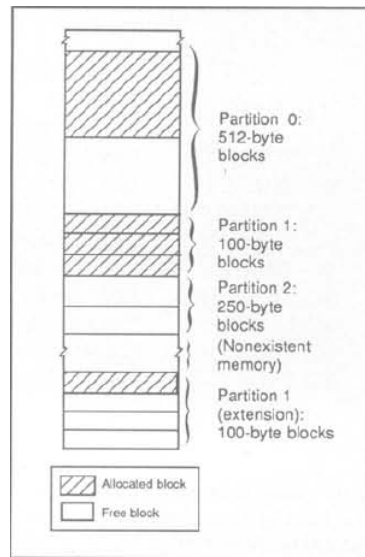


Figure 2.4: Memory management in VRTX (Extracted from [4])

In contrast, general-purpose OSes allow applications to heavily utilize dynamic memory allocation with variable-sized blocks and virtual memory with paging. These methods offer flexibility and efficient memory utilization but can lead to unpredictable timing delays.

When a program in a general-purpose OS allocates memory dynamically (e.g., using `malloc` in C), the OS searches the process's heap for a suitable block. Over time, as processes allocate and free memory of different sizes, the heap can become fragmented, making allocation times variable and unpredictable. Additionally, general-purpose OSes commonly use virtual memory. If a process tries to access a memory page that is not currently in physical RAM, a page fault occurs, requiring the OS to load the page from slower storage (like a hard drive or SSD). This can cause a significant and unpredictable delay, incompatible with the strict timing requirements of RTOSes.

3 Scheduling

As explained previously, real-time systems follow rigid scheduling policies so that tasks are executed within their necessary time constraints. Real-time tasks have deadlines to meet, and it is essential to estimate the Worst-Case Execution Time (WCET) of a task to guarantee deadline compliance even under worst-case loads. Nevertheless, some leniency in the execution time is always necessary based on the criticality of the task as well as deadline tolerance.

Scheduling is also based on the nature of the task—whether periodic, at fixed intervals, or aperiodic, at arbitrary intervals. The scheduling algorithm must factor these in for it to effectively manage resources and satisfy timing constraints.

In this chapter, we will cover the scheduling algorithms that have been devised to handle real-time constraints and their performance in different scenarios. We'll also learn how fault tolerance can be incorporated into task scheduling so that the system remains dependable even in case of task or system failures. Finally, we'll learn how system architecture influences scheduling, including how hardware and software components must be balanced for optimal performance. By the time we finish this chapter, we will know more about how to develop effective and versatile scheduling mechanisms that address true real-time needs and tolerate task failures.

3.1 Different types of scheduling

Scheduling algorithms can be preemptive or non-preemptive. Preemptive algorithms allow a running task to be interrupted and resumed later, which is often necessary for predictable real-time scheduling with acceptable processor utilization. Non-preemptive algorithms have less context switching overhead but force stricter limitations on the scheduler.

All real-time scheduling algorithms rely on priorities. The task instance with the highest priority among all active instances is executed. Priorities are given in two different ways, either at the beginning of the tasks (static or fixed priority) or during runtime. I am going to be presenting one algorithm for each of the ways.

3.1.1 Rate Monotonic Priority Assignment(RM)

Rate Monotonic (RM) is a static priority scheduling algorithm. Priorities are assigned prior to the execution and do not change at runtime. RM assigns priorities on the basis of the period of periodic tasks: the shorter the period, the higher the priority (higher activation rate). RM is preemptive in nature.

Figure 3.1 illustrates the RM scheduling strategy with two tasks τ_1 (precisely 7 instances of the task) and τ_2 (precisely 5 instances of the task). Each instance of the task τ_i is here defined by its relative deadline (period) D_i and its WCET C_i . For τ_1 :

$D_1 = 5, C_1 = 1$ and for τ_2 : $D_2 = 7, C_2 = 4$. At time 15, we can see that the third instance of τ_2 is preempted by the fourth instance of τ_1 which shows that RM assigns higher priority to shorter tasks because τ_1 has a smaller WCET and smaller period than τ_2 . The same thing happens at time 30.



Figure 3.1: Example of RM on multiple instances of τ_1 and τ_2 (Extracted from [1])

RM is primarily used for periodic tasks because its priority assignment is based on the fixed, known periods of these tasks. For tasks with irregular or unknown arrival times (aperiodic tasks), assigning a static priority based on a period is not applicable. While aperiodic tasks can be handled in systems using RM, it is typically done through server mechanisms which manage the aperiodic requests and present them to the RM scheduler as if they were periodic.

One important aspect of real-time scheduling is schedulability analysis, which determines whether a task set is schedulable to meet all deadlines. For RM, a simple test is to compare the utilization of the task set ($U = \sum_{i=1}^n \frac{C_i}{D_i}$ with C_i the WCET and D_i the relative deadline of the task) with a least upper bound (LUB) of utilization, $U_{lub} = n(2^{\frac{1}{n}} - 1)$, which tends to $\ln 2 \approx 0.69$ as n gets large. If $U \leq U_{lub}$, the task set is RM-schedulable. By utilization, we are talking about how much a certain task set uses the CPU. The LUB is what utilization a set of tasks can achieve and we are easily sure it can be scheduled. In [6], in chapter 4, the LUB is explained further and in [11] its value is proven. The test done for RM is sufficient but not necessary, though, because task sets with $U_{lub} < U < 1$ may still be schedulable.

A more effective test for fixed-priority scheduling algorithms such as RM (and Deadline Monotonic, DM) is the Response Time Analysis. It calculates the worst-case completion time (response time, R_i) for each task τ_i . The set is schedulable if $R_i \leq D_i$ for every task. The response time calculation is a recursive process considering the task's WCET and interference from higher priority tasks.

The utilization bounds like $n(2^{\frac{1}{n}} - 1)$ for RM represent a schedulability limit. If the total utilization of a task set exceeds this bound, the RM algorithm cannot guarantee that all deadlines will be met, even if the processor is not fully utilized ($U < 1$). This is a key difference compared to optimal dynamic priority algorithms like Earliest Deadline First (EDF), which can schedule any task set on a single processor if the total utilization is less than or equal to 1, ($U \leq 1$). The lower utilization bound of RM for $n > 1$ is a consequence of its static priority assignment, which can lead to situations where a higher-priority task with a long period unnecessarily blocks a lower-priority task with a close deadline.

3.1.2 Earliest Deadline First Scheduling (EDF)

Earliest Deadline First (EDF) is a dynamic priority scheduling algorithm. The priorities are assigned on the basis of the absolute deadline of task instances: the closer the absolute deadline, the higher the priority. It means priorities can change at runtime. Like RM, EDF is preemptive in nature.

Figure 3.2 illustrates the EDF strategy with the same parameters as in the RM example. At time 15, τ_2 is preempted by τ_1 because the relative deadline of τ_1 is closer ($15 + 5 = 20$, 15 being the arrival time and 5 being the relative deadline) than the one of τ_2 ($14 + 7 = 21$). However, unlike in the RM example, at 30 there is no preemption since we have the same absolute deadline ($28 + 7 = 30 + 5 = 35$) and the execution of τ_2 ends early enough to allow the execution of τ_1 .

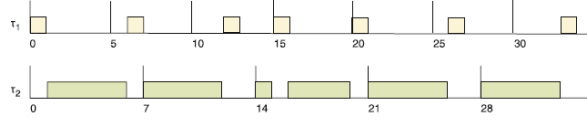


Figure 3.2: Example of EDF on multiple instances of τ_1 and τ_2 (Extracted from [1])

A simple and exact schedulability test for EDF is the utilization test: a task set is schedulable by EDF if and only if its total utilization $U = \sum_{i=1}^n \frac{C_i}{D_i} \leq 1$.

EDF can also be applied to aperiodic task sets and minimizes the maximal lateness ($L_{i,j} = f_{i,j} - d_{i,j}$).

In spite of its optimality and easier test of schedulability, fixed-priority algorithms such as RM are still favored in practice because of perceived implementation complexity of dynamic priorities and fears of the “Domino Effect” under overload conditions in EDF (although there have been works such as in [12] indicating that these fears may be less well-founded in practice).

3.1.3 Servers for aperiodic tasks

To handle aperiodic tasks within a framework primarily designed for periodic tasks, server mechanisms can be used. A server is typically modeled as a periodic task that is allocated processor time, which it then uses to execute any pending aperiodic tasks. This allows aperiodic tasks, which do not have known arrival times, to be integrated into scheduling schemes like RM that are fundamentally designed for periodic workloads. Examples of such servers include Polling Servers, Deferrable Servers, and Sporadic Servers. Dynamic priority servers like the Total Bandwidth Server (TBS) and Constant Bandwidth Server (CBS) also exist for use with EDF scheduling. These servers aim to improve the responsiveness of aperiodic tasks while ensuring the schedulability of periodic tasks.

3.1.4 Why RM and EDF?

There are a lot of real-time scheduling algorithms that exist besides RM and EDF. However, the list of those algorithms is very long. That is why in this chapter I have only presented RM and EDF. Each one, as mentioned previously, represents a type of priority assignment by real-time schedulers. You may still be wondering why RM and EDF though. RM has been proven as optimal among all fixed priority scheduling algorithms in [11]. It means that if a set of tasks is schedulable by RM, it is schedulable by any fixed priority scheduling algorithm. EDF has been proven optimal among all dynamic priority scheduling algorithms in [11]. It means that if a set of tasks is schedulable by EDF, it is schedulable by any dynamic priority scheduling algorithm. Thus, it is a very important algorithm to understand in the context of real-time.

3.2 Fault-tolerance

A real-time system might benefit from fault-tolerance. Sometimes, it is important to have a system that will not go completely off the rails due to unexpected faults. Fault-tolerance in real-time systems, in contrast to normal OSes, is paramount because missing a deadline due to a fault can have severe, even catastrophic, consequences. While general-purpose OSes might detect and log errors or restart applications after a crash, real-time systems often require the system to continue operating predictably even in the presence of faults. This is often achieved through redundancy (like time redundancy) and specific recovery mechanisms integrated into the scheduling.

The main problem with faults is the obligation to restart a task from the beginning if it fails. In [13], a proposition is made for such a problem in the seL4 Microkernel. The objective is to add a checkpoint mechanism to remediate the problem. In the scheduling of a task, there will be a time to save the context of the thread executing it. It means, if the execution was to ever fail, the restart would only happen at the last save which is called a checkpoint. This is possible because the thread context and stack space are saved, which means we have access to the last state of the task.

This approach aims to decrease the overhead of error recovery by avoiding re-execution from the very beginning of a task. This is a crucial aspect of maintaining predictability and meeting deadlines in fault-prone real-time environments.

3.3 Architecture-awareness

The increasing computational demands of modern real-time embedded systems necessitate increased performance, but they make scheduling for real-time harder. Unlike single-core systems, where task execution time can often be characterized by a single Worst-Case Execution Time (WCET), the actual execution time of a task on a multi-core platform is highly dependent on the interference it experiences from other cores accessing shared resources. This variability makes guaranteeing deadlines difficult and unpredictable. This has led to the development of architecture-aware scheduling approaches

that explicitly consider the underlying hardware to ensure predictability and optimize performance.

In [14], the authors propose the integration of programmable logic (PL), such as FPGAs, within MPSoCs to manage critical resources and enable predictable communication. By using hardware-software co-design, they aim to achieve isolation between different criticality domains, preventing interference between critical and non-critical tasks. At the hardware level, dedicated scratchpad memories (SPMs, high-speed CPU-internal memory used for temporary storage of work in progress) with controlled interfaces within the PL can be used to avoid conflicts in the main memory. Software-level techniques, such as hypervisors supporting cache-coloring, can mitigate conflicts in shared caches like the L2 cache.

Direct Memory Access (DMA) inspired controllers assure efficient communication between main memory and SPMs while moving tasks. The scheduling of these controllers' transfer, in conjunction with CPU execution, is crucial for reducing memory latency and maximizing processor utilization. The three-phase task model (Load, Computation, Unload) explicitly models this by separating memory access phases (Load/Unload) from the execution phase (Computation). SPMs can be divided into partitions, allowing one task to execute from one partition while the controller loads or unloads data for another task in a different partition.

Furthermore, the scheduling policy for tasks and their memory phases impacts the overall system schedulability. Simply starting the next task's load phase as soon as the current task's computation begins (Eager Load) can lead to increased blocking for higher-priority tasks if a lower-priority task's load operation is already in progress. The Lazy Load scheduling policy addresses this by postponing the decision to schedule the controller load for the next task as late as possible, specifically a certain number of time units before the end of the current task's computation phase, aiming to overlap the memory transfer with the remaining computation time. This approach reduces blocking from lower-priority tasks and improves system schedulability.

The authors implement their design on the Xilinx UltraScale+ ZCU102 MPSoC. However, their implementation is a little bit tricky. As we are talking about hardware design, some changes need to be made, and in addition to that, the MPSoC they use does not have SPMs but Block RAM (BRAM), which is a slightly different type of high-speed memory. In their implementation, they use BRAM as SPMs. They separate it into three different parts, each one assigned to a criticality domain, and each part is separated into two partitions. They use the DMA in a slightly different way than intended, for transferring data from caches to SPMs (BRAM here). The rest is implemented as explained in their paper. Their implementation was evaluated by benchmarking the processing of images with their MPSoC. The evaluation shows increased predictability. This shows that it is possible to implement their design; it is not just theory.

The development of scheduling policies like Lazy Load highlights a shift towards optimizing the overlap between computation and communication/memory access, rather than simply minimizing individual phase durations. On modern architectures, the way tasks interact with shared resources is as important as their individual execution times. By delaying scheduling decisions and leveraging knowledge of task execution progress, the scheduler can make more informed choices that reduce priority inversion and improve

overall system predictability.

Beyond explicit utilization of particular hardware aspects and fixed policies, other methods explore the potential of scheduling based on task characteristics as well as the architecture that it will run on. For example, work done in [15] has explored how to utilize deep reinforcement learning and graph convolutional networks toward creating scheduling approaches for intricate task dependencies in multiprocessor systems with the aim of enhancing performance metrics such as reductions in makespan. This approach was evaluated using synthetic datasets and a task scheduling simulator, demonstrating performance improvements compared to other heuristics in various configurations of processors and task parallelism. This highlights a shift towards data-driven methods that implicitly capture hardware-related factors influencing task execution and interaction to improve scheduling decisions.

This approach goes beyond the three-phase model in [14], exploring more precise execution models that capture the time-varying resource usage characteristics of tasks under different shared resource allocations.

4 Optimizing code generation

Optimization in compilers is valuable for generating efficient code, but in real-time systems, their application meets special challenges. The basic problem is to produce code that not only runs well on the average but, most significantly, meets hard real-time requirements, e.g., meeting deadlines and respecting Worst-Case Execution Time (WCET) constraints. Traditionally, ensuring compiled code met these constraints often defaulted to manual testing or analysis, which became almost impossible because of modern system complexity. Traditional compiler optimizations for average-case performance can inadvertently increase WCET or make it significantly harder to estimate, making verification more challenging. An example of such optimization is the function inlining optimization. This optimization replaces function calls by their code, which reduces the overhead of a function call. However, it increases the code size, which subsequently increases the risk of missed caches. This is taken into consideration for WCET analysis, and the estimated WCET is increased. The intricate interaction of compiler-generated code, complex application logic, software stacks, and Real-Time Operating System (RTOS) scheduling only makes it further difficult to verify temporal behavior upon compilation.

Furthermore, predictability and usefulness of compiler optimizations also heavily rely on the intended embedded architecture. Optimal generation of code requires deep knowledge about processor pipelines, memory hierarchies, caching interactions, and peripheral communication. But optimizations which are good for a particular architecture will have varying, potentially unpredictable timing effects on another.

This inevitably implies that there is a real need for closely coupled automated solutions. Compilers have to shift away from traditional performance goals towards incorporating knowledge of real-time. This involves the incorporation of static analysis techniques for WCET estimation, the development of novel optimization passes directed towards timing predictability (often even to the detriment of average performance), and the generation of code that predictably interacts with the host RTOS and the hardware.

In this chapter, I will present existing compiler techniques, associated tools, and techniques developed to overcome these obstacles in verifying the temporal behavior of compiled code for real-time embedded systems. I will explain how compilers can be extended or re-engineered to assist programmers in ensuring the generated code meets imposed constraints. In addition, the chapter will touch on the strong potential for future research, citing open research topics.

4.1 The Need for WCET-Aware Compilation

In this section, I will be explaining why we need WCET-Awareness in compilation, the known solutions, and how they are achieved.

4.1.1 The Role of Control Flow Information

Optimizing code while taking into account WCET heavily relies on flow information. If we want to be able to improve the WCET of a program, we have to get information about how individual statements, instructions, and function calls are going to be evaluated. It means getting their order and how they depend on each other.

The most common representation of flow information is a directed graph. Such a graph shows the relation between different blocks. Each node represents a block, and the link between nodes allows us to have the different possible paths of the execution of the code. We call that graph a control flow graph (CFG) as mentioned in [16]. These graphs are very commonly found in most WCET analysis results and used for calculations. It is very important today in most analyses, not only WCET.

4.1.2 Managing Control Flow Information During Optimization

Using control flow in WCET calculations was introduced in the 1990s to the 2000s. A co-transformation of flow information was proposed in [17] and [18] to be able to track the influence of optimization on flow information. The core idea was to preserve and adapt this high-level flow information throughout the compiler's optimization stages. This required analyzing execution information derived from the high-level source code (like loop bounds) in parallel with the low-level code transformations performed by the compiler (e.g., instruction scheduling, register allocation, function inlining). The co-transformer's intended role was then to map the high-level flow information (like the source-level loop structure and its known maximum iteration count) onto the corresponding, potentially heavily modified, structures in the optimized low-level code (assembly or machine code). However, the co-transformer was never fully functional. It does not work with all standard optimizations because of the limitation of the proposed data structure. It cannot capture every necessary detail, such as complex loop transformations which can lead to bad calculations of WCET. An example of a complication would be that techniques like function inlining (replacing a function call with the function's body) or aggressive instruction scheduling could reorder code blocks or eliminate previously existing control flow paths, further complicating the task of accurately mapping source-level flow constraints. In [19], a method to resolve that issue is proposed: instead of having a tool doing the mapping, it is done by the compiler. That is the first step towards WCET-Aware Compilers. Despite that, it still is limited to a subset of the C language. In [20] and [21], the integration in the compiler is proposed. While being efficient, it only covers loops and functions and not simple statements. We will focus on WCET-Aware C Compiler (WCC) in the next subsection, which is the first of its kind.

Even though we could analyze and calculate WCET by that time, the control flow has continued to be fascinating for real-time embedded systems. In [22] and [23], traceability of flow information in order to calculate WCET was proposed. From source code to binary level the flow information is followed through all the transformations before being used for WCET estimate. The first proposition was a prototype traceability tool over LLVM, but has unfortunately never been integrated into it. The second proposition was the use of that tool to tighten the WCET calculation.

It has also been discovered that flow information can be a source for attackers to exploit. In microcontroller-based systems, often written in C, memory errors can allow attackers to overwrite critical control data like return addresses, function pointers, and saved processor state. This is particularly dangerous because application tasks and the OS kernel typically share the same memory space, meaning one vulnerability can compromise the entire system. By corrupting control data, attackers hijack the intended execution flow. In [24], a control-flow defense software, Kage, implementation in FreeRTOS is proposed. It mitigates the previously mentioned risks by storing control data in protected memory regions separate from untrusted data. Insecure code is restricted from directly accessing this protected memory and must communicate through a secure API. This design limits the extent to which an attacker can manipulate the control flow of the system and aims to make useful code-reuse attacks impractical. Kage focuses on protecting against code-injection and code-reuse attacks.

4.1.3 WCET-Aware Compiler Design and Techniques

Tools for WCET calculations and analyses such as WCET analyzers already existed before their integration into compilers. They are still important outside of their compiler use. Moreover, modern safety-critical certifications require WCET analysis to guarantee timing deadlines are always met. Examples are the DO-178C (software development standard for airborne systems) and ISO 26262 (international standard for functional safety in the automotive industry). In the DO-178C (see [25]), for example, the software accomplishment summary requires timing margins including WCET. In ISO 26262 (see [26]), for example, software architectural design requires the inclusion of properties such as execution time (which naturally includes WCET bounds). In these cases, results of a WCET analyzer can be important.

The most used WCET analyzer in the industry is aiT. It is the first one developed for such a task. It decodes CFGs, analyses of address ranges to be accessed, loop bounds, cache, pipelines, and paths as mentioned in [27]. This allows the WCET calculation to take into account how many iterations will be done, how much time it takes to access memory addresses, and what is the worst possible path of a program. In [28] and [29] open-source alternatives, OTAWA and Chronos, to aiT are shown even though the articles do not mention them as alternatives but present them by themselves. OTAWA is a modular tool which is designed to be platform-independent. Like aiT, it is also used for verifications required by safety-critical norms although it is not as popular. Its components can be used by researchers in other WCET analyzers or even be combined with other analyzers. Chronos, on the other hand, is not commonly used for norm compliance but mostly in research settings.

With the development of WCET calculations and methodologies to manipulate flow information, the design for WCC was introduced in 2006 [30]. WCC goes first through classic compiler methods which will provide low-level intermediate representation (IR) through the process. Such a low-level representation takes into account processor-specific information. At that level, a WCET analysis is done. The analysis can only be done after the code has been represented at low-level due to the inclusion of processor-specific information. The analysis is done by aiT. While optimizing the code, there is an exchange between the analyzer and the component for optimization. WCC is trying to make sure

optimizations do not compromise the WCET.

Early efforts in Worst-Case Execution Time (WCET) aware compilation included standalone compilers like WCC , which was specifically designed for WCET minimization. However, integrating WCET-awareness directly into mainstream compilers such as LLVM presents significant challenges. This is partly because general-purpose compilers typically prioritize average-case performance optimizations, which can sometimes conflict with the goal of achieving predictable worst-case timing.

While being able to optimize WCET in code, it is sometimes still interesting to keep some general-purpose optimization. That would mean that we need a compiler that can accomplish multiple objectives. We are introduced to the term multiobjective optimization.

4.2 Multiobjective optimization

How do we get compilers that optimize not only WCET but also other objectives needed by a real-time system such as Average-Case Execution Time, code size, or even power consumption? We have seen compilers becoming more WCET-aware, but is it possible to keep some of the needed classic compiler optimizations?

Such an optimization is particularly challenging. The main principle is finding a way to have multiple conflicting objectives met without constraining one another. Such a problem is met in other fields too. This is where the concept of Pareto optimality is met (See Appendix A). We need a set of solutions that are non-dominated by others while still being better than other solutions. We call them Pareto fronts. In [31] was proposed the first WCET-Aware compiler framework to automatically search for the solutions. The authors focus on three objectives : WCET, ACET (Average Case Execution Time) and code size. Evolutionary Multi-Objective (EMO) Algorithms, such as SPEA2 (discussed in Appendix B), are employed to tackle this by approximating the *Pareto front*. These algorithms work by simulating the process of natural selection over a population of potential compiler solutions. This simulation typically involves three core mechanisms:

- **Selection:** Solutions that demonstrate better performance across the various optimization objectives are preferentially chosen to survive and reproduce. This drives the population towards more optimal regions of the solution space.
- **Mutation:** Small, random modifications are introduced to some solutions. This helps maintain diversity within the population and allows the algorithm to explore new, potentially beneficial areas of the search space that might not be reachable through selection and crossover alone.
- **Crossover:** Genetic information (like compiler flag combinations or optimization strategies) from two or more selected “parent” solutions is combined to create new “offspring” solutions. This mechanism allows the algorithm to inherit and combine desirable traits from successful solutions, potentially leading to even better trade-offs.

By iteratively applying these steps – selecting good solutions, introducing variation through mutation, and combining features through crossover – EMO algorithms effectively search for and converge upon the set of Pareto optimal solutions, providing the compiler designer with a diverse range of high-quality trade-offs to consider.

Multi-objective exploration of compiler optimizations has shown significant improvements compared to standard optimization levels. According to [31] the discovered optimization sequences can significantly outperform commonly used standard optimization levels. For instance, the highest standard optimization level O3 was outperformed for the Worst-Case Execution Time (WCET) and Average-Case Execution Time (ACET) on average by up to 31.33% and 27.43%, respectively. The evolutionary search also proved beneficial over generations; while the first generation achieved WCET and ACET reductions of 36.9% and 35.0% on average, the 50th generation reduced WCET and ACET by up to 42.9% and 42.8%, respectively.

Multi-objective optimization in compiler design often produces a very large set of potential solutions, potentially numbering in the thousands to tens of thousands (or even more when considering the vast combinatorial space of compiler flags). Each solution represents a different configuration or set of trade-offs, for instance, balancing code execution speed against memory footprint. For a compiler designer, effectively analyzing and selecting from this extensive set of, say, 5,000 to 20,000 potential optimal configurations is a significant challenge due to its sheer size. To make this process tractable, it is beneficial to group similar solutions together into smaller, more manageable sets.

[32] proposes a clustering approach specifically designed to address this challenge by imposing a size limit on the resulting clusters. This method, detailed in Appendix C (specifically Algorithms 2, 3, and 4), transforms the large initial set of solutions into a collection of clusters, where each cluster contains similar solutions and adheres to a predefined maximum size, denoted by τ .

The overall clustering process (Algorithm 2) involves two primary phases after an initial clustering step:

1. Refining Large Clusters (Algorithm 3): An initial clustering might produce some clusters that still exceed the desired maximum size τ . The refinement phase iteratively identifies the largest cluster and re-clusters it into smaller sub-clusters. This process is repeated until all clusters are guaranteed to have a size less than or equal to τ . This step ensures that no single resulting group is too large for a designer to effectively analyze.
2. Merging Small, Close Clusters (Algorithm 4): After refinement, there might be numerous small clusters. This phase aims to reduce the total number of clusters by merging those that are geographically close to each other in the solution space (i.e., the distance between their centroids or closest points is less than a threshold $dist_{max}$). A crucial condition for merging is that the combined size of the two clusters must not exceed the maximum size τ . This step consolidates similar solutions into larger groups where possible, without violating the size constraint, thus providing a more condensed overview of the solution landscape.

By applying these two constrained clustering steps, the overwhelming initial set of

solutions is reduced to a reasonable number of clusters, each of a manageable size. This structure significantly simplifies the compiler designer’s task, allowing them to analyze representative solutions within each cluster and make more informed decisions regarding compilation optimization strategies based on the grouped trade-offs.

4.3 Hardware-awareness

There is an increasing demand for high performance, energy efficiency, and predictability in real-time systems which inevitably involves parallelism. Although we have seen some scheduling methods on such systems in chapter 3, we can notice that it is still highly difficult to have accurate predictions on resource allocation management. We do not always know which core is doing what.

Let’s take a look at one of the most adopted multi-core technologies in recent years, Multiprocessor Systems-on-Chips (MPSoCs), and what is proposed to solve issues at the compilation-level. An MPSoC compiler is a toolchain designed to address application modeling, platform description, software parallelization, software distribution, and code generation for efficient use of the target platform as described in [5]. As detailed in Table 4.1, MPSoC compilers extend traditional single-core compilation by tackling these additional complexities inherent in parallel and heterogeneous environments. The transition to MPSoCs significantly raises the complexity for compilers. For instance, while single-core compilers require architecture information like the instruction set for the backend, MPSoC compilers need more extensive platform descriptions, including details about Processing Elements (PEs) and communication resources, used across multiple compilation phases. Software parallelization also differs: single-core compilers focus on Instruction-Level Parallelism (ILP), whereas MPSoC compilers target coarser-grained parallelism as explained in [5]. Consequently, software distribution involves assigning coarse-grained tasks in MPSoC compilation, unlike the instruction-level distribution in single-core compilers [5]. Code generation for heterogeneous PEs in MPSoCs is also markedly more complex than for a single instruction-set architecture [5].

In the context of real-time systems, achieving predictable behavior is key [5]. While single-core compilation for real-time systems has established methods for Worst-Case Execution Time (WCET) analysis, multi-core compilation for real-time MPSoCs introduces substantial challenges. Ensuring predictability involves complex scheduling, synchronization, and managing shared resources, making WCET analysis and guaranteeing real-time performance significantly harder due to inter-core interference and platform heterogeneity [5]. Performance estimation techniques, which are crucial, include analytical, emulation-based, simulation-based, and table-based methods, each with its own complexities, especially in the presence of features like caches [5].

Table 4.1: Comparison of Single-Core and Multi-Core Compilation Based on [5]

Feature	Single-Core Compilation	Multi-Core Compilation (MPSoC)
Primary Goal	Translate high-level language to machine binary for one core [5].	Tackle application modeling, platform description, software parallelization, distribution, and code generation for an MPSoC platform [5].
Complexity	Well-understood and practiced for various core types (RISC, DSP, VLIW) [5].	Orders of magnitude more complex; faces open issues in academia and industry [5].
Programming Models Input	Typically sequential languages (e.g., C/C++) which hide hardware details [5].	Needs to cope with sequential legacy code and parallel programming models to efficiently utilize hardware resources [5].
Platform Description	Requires architecture info (e.g., instruction set, latency table) for the backend [5].	Needs more detailed platform description (PEs, communication resources, memory subsystem) used in multiple compilation phases, not just the backend [5].
Software Parallelization Focus	Exploits Instruction-Level Parallelism (ILP) [5].	Focuses on a wider variety of more coarse-grained forms of parallelism [5].
Software Distribution	Performs this at instruction-level [5].	Distributes coarse-grained tasks (or code blocks); involves mapping tasks to PEs and scheduling [5].
Code Generation	Generates binary for a one-ISA architecture [5].	"Leaping complexity" to generate final binaries for heterogeneous PEs and Network-on-Chip (NoC) [5]. Often ad-hoc due to heterogeneity [5].
Compiler Structure (General Phases)	Front end, Middle end, Back end [5].	Software Parallelization, Software Distribution, Code Generation [5].
Real-time Performance & Predictability	Meeting real-time constraints is crucial; WCET analysis is relatively more established.	Predictable behavior is key but harder to achieve; WCET analysis is difficult due to caches, non-deterministic features, and inter-core interference [5].

5 Synthesis of Concepts and Applications: A Tabular Overview

The following table (5.1) shows which of the concepts are actually very commonly used in RTOSes. As you will notice, some of the more complex features are not represented because they are sometimes directly applied to the architecture (e.g. MPSoC) but the general algorithms and some compiler features are represented.

Table 5.1: A synthesis of the real-time concepts applied in three RTOSes

Feature	FreeRTOS	QNX	RTEMS
Scheduling	Priority-based preemptive scheduler. Rate Monotonic Analysis (RMA) and Deadline Monotonic Analysis can be used, often with extensions for aperiodics and shared resources. Can be extended with EDF schedulers.	Priority-driven preemptive scheduling. Adaptive Partition Scheduling (APS) guarantees minimum CPU percentages to groups while remaining priority-preemptive under normal load. Highest priority runnable thread always runs on an available core.	Pluggable scheduling framework. Supports: Deterministic Priority (with preemption), Simple Priority (lower RAM, $O(n)$), Rate Monotonic (RMS), Earliest Deadline First (EDF), Constant Bandwidth Server (CBS), Round Robin schedulers. Priority-based preemption and time-slicing are common configurations.
Compilers	Supports various C compilers depending on the target architecture (e.g., GCC, ARM Compiler). WCET-aware C Compiler (WCC) efforts and LLVM infrastructure use for WCET analysis.	Primarily uses GCC. Supports other compilers compatible with its supported architectures.	Primarily uses GCC (included in the RTEMS Tool Suite). Supports cross-compilation.
Uses	Widely used in embedded systems, IoT devices, medical devices, automotive ECUs, consumer electronics. Particularly popular for smaller embedded systems.	Automotive (infotainment, ADAS), medical devices, industrial control systems, robotics, trains, network infrastructure, embedded systems requiring high reliability.	Space systems (satellites, space probes - supports SPARC, LEON, PowerPC), embedded systems with stringent real-time requirements, scientific applications, medical devices, industrial control. Supports many processor architectures.

6 Conclusion and reflection

An initial study of real-time systems and their associated compiler challenges might suggest a field where the fundamental problems are largely solved. The common discourse around real-time systems often overlooks their underlying complexity. Ensuring precise timing is significantly more challenging than it appears, particularly as hardware evolves. Standard operating systems cannot guarantee adherence to strict deadlines, necessitating RTOSes. These employ specialized techniques, like deterministic scheduling, to address timing requirements, yet an RTOS alone is not a complete solution.

This survey highlights the compiler’s critical function, extending beyond simple code translation to that of an essential intermediary cognizant of the strict constraints within real-time environments. The investigation covered the necessity for compiler optimizations to preserve timing integrity (WCET-awareness) and the requirement for compilers to deeply understand the target hardware (hardware-awareness). It is evident that achieving reliable real-time operation hinges not just on the OS or the hardware independently, but on their intelligent interaction, largely orchestrated by the compiler. The challenge of optimizing for multiple, often competing, goals such as performance, predictability, and energy efficiency simultaneously underscores the sophistication needed in contemporary compiler design for these systems.

Initial perceptions, perhaps influenced by the volume of foundational literature, might underestimate the ongoing dynamism and relevance of research in this domain. Such perceptions, however, prove inaccurate. Emerging challenges, including the proliferation of multi-core processors, system heterogeneity, and the integration of demanding applications like machine learning into embedded contexts, have intensified the field’s complexities and importance. Predictability, more readily achievable in simpler single-core architectures, has re-emerged as a central and difficult research question in the multi-core landscape.

This evolving context presents compelling directions for future work. The application of deep learning or even machine learning to WCET analysis and system resource management might hold some potential, possibly offering predictive capabilities that surpass traditional static analysis in handling the dynamic nature of multi-core systems. Could deep learning or machine learning techniques help recover a measure of predictability diminished by parallel hardware complexities by learning and adapting to system behaviors? Concurrently, continued innovation in hardware/software co-design, aiming to embed properties like temporal and spatial isolation directly into the system architecture, remains crucial. Enabling the reliable coexistence of tasks with varying criticality levels on shared hardware demands fresh approaches to processor and software stack design.

Ultimately, this body of work underscores the compiler’s indispensable role in engineering dependable real-time systems. The associated challenges are not static; they are transforming, requiring compilers that exhibit greater intelligence, contextual sensitivity, and hardware insight. The ongoing pursuit of predictability, performance, and efficiency amidst increasing system complexity ensures that the synergy between operating systems, compilers, and hardware architecture will continue to be a vital and productive area for research and innovation.

References

- [1] F. Rammig, M. Ditze, P. Janacik, T. Heimfarth, T. Kerstan, S. Oberthuer, and K. Stahl, “Basic Concepts of Real Time Operating Systems,” in *Hardware-dependent Software: Principles and Practice*, W. Ecker, W. Müller, and R. Dömer, Eds. Dordrecht: Springer Netherlands, 2009, pp. 15–45. [Online]. Available: https://doi.org/10.1007/978-1-4020-9436-1_2
- [2] F. Guan, L. Peng, L. Perneel, and M. Timmerman, “Open source freertos as a case study in real-time operating system evolution,” *Journal of Systems and Software*, vol. 118, pp. 19–35, 2016.
- [3] T. D. Juhász, S. Pletl, and L. Molnar, “A Method for Designing and Implementing a Real-Time Operating System for Industrial Devices,” in *2019 IEEE 13th International Symposium on Applied Computational Intelligence and Informatics (SACI)*, May 2019, pp. 149–154. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9111571>
- [4] J. Ready, “VRTX: A Real-Time Operating System for Embedded Microprocessor Applications,” *IEEE Micro*, vol. 6, no. 4, pp. 8–17, Aug. 1986. [Online]. Available: <http://ieeexplore.ieee.org/document/4089679/>
- [5] R. Leupers, M. A. Aguilar, J. Castrillon, and W. Sheng, “Software Compilation Techniques for Heterogeneous Embedded Multi-Core Systems,” in *Handbook of Signal Processing Systems*, S. S. Bhattacharyya, E. F. Deprettere, R. Leupers, and J. Takala, Eds. Cham: Springer International Publishing, 2019, pp. 1021–1062. [Online]. Available: https://doi.org/10.1007/978-3-319-91734-4_28
- [6] “Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications - Second Edition, Springer 2005.” [Online]. Available: https://www.researchgate.net/publication/246357308_Hard_Real-Time_Computing_Systems_Predictable_Scheduling_Algorithms_and_Applications_-_Second_Edition_Springer_2005
- [7] D. E. Culler, J. Hill, P. Buonadonna, R. Szewczyk, and A. Woo, “A network-centric approach to embedded software for tiny devices,” in *Embedded Software: First International Workshop, EMSOFT 2001 Tahoe City, CA, USA, October 8–10, 2001 Proceedings 1*. Springer, 2001, pp. 114–130.
- [8] Y. Chen, X. Tang, S. Xu, F. Zhu, Q. Zhou, and T.-H. Weng, “Analyzing execution path non-determinism of the linux kernel in different scenarios,” *Connection Science*, vol. 35, no. 1, p. 2192442, 2023.
- [9] A. Venkataraman and K. K. Jagadeesha, “Evaluation of inter-process communication mechanisms,” *Architecture*, vol. 86, no. 64, 2015.
- [10] A. Damm, J. Reisinger, W. Schwabl, and H. Kopetz, “The real-time operating system of MARS,” *SIGOPS Oper. Syst. Rev.*, vol. 23, no. 3, pp. 141–157, Jul. 1989. [Online]. Available: <https://dl.acm.org/doi/10.1145/71021.71029>

- [11] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM (JACM)*, vol. 20, no. 1, pp. 46–61, 1973.
- [12] G. C. Buttazzo, “Rate monotonic vs. edf: Judgment day,” *Real-Time Systems*, vol. 29, pp. 5–26, 2005.
- [13] L. Xu, Y. Bai, K. Cheng, L. Ge, D. Nie, L. Zhang, and W. Liu, “Towards Fault-Tolerant Real-Time Scheduling in the seL4 Microkernel,” in *2016 IEEE 18th International Conference on High Performance Computing and Communications; IEEE 14th International Conference on Smart City; IEEE 2nd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, Dec. 2016, pp. 711–718. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7828445>
- [14] T. Kloda, G. Gracioli, R. Tabish, R. Mirosanlou, R. Mancuso, R. Pellizzoni, and M. Caccamo, “Lazy Load Scheduling for Mixed-criticality Applications in Heterogeneous MPSoCs,” *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 3, pp. 59:1–59:26, May 2023. [Online]. Available: <https://doi.org/10.1145/3587694>
- [15] H. Lee, S. Cho, Y. Jang, J. Lee, and H. Woo, “A global dag task scheduler using deep reinforcement learning and graph convolution network,” *IEEE Access*, vol. 9, pp. 158 548–158 561, 2021.
- [16] F. E. Allen, “Control flow analysis,” *SIGPLAN Not.*, vol. 5, no. 7, pp. 1–19, Jul. 1970. [Online]. Available: <https://dl.acm.org/doi/10.1145/390013.808479>
- [17] J. Engblom, “Worst-case execution time analysis for optimized code,” in *In Proceedings of the 10th Euromicro Workshop on Real-Time Systems*. Citeseer, 1997, pp. 146–153.
- [18] J. Engblom, A. Ermedahl, and P. Altenbernd, “Facilitating worst-case execution times analysis for optimized code,” in *Proceeding. 10th EUROMICRO Workshop on Real-Time Systems (Cat. No.98EX168)*. Berlin, Germany: IEEE Comput. Soc, 1998, pp. 146–153. [Online]. Available: <http://ieeexplore.ieee.org/document/685079/>
- [19] R. Kirner and P. Puschner, “Transformation of path information for WCET analysis during compilation,” in *Proceedings 13th Euromicro Conference on Real-Time Systems*, Jun. 2001, pp. 29–36. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/933993>
- [20] W. Zhao, P. Kulkarni, D. Whalley, C. Healy, F. Mueller, and G.-R. Uh, “Timing the WCET of embedded applications,” in *Proceedings. RTAS 2004. 10th IEEE Real-Time and Embedded Technology and Applications Symposium, 2004.*, May 2004, pp. 472–481, iSSN: 1545-3421. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1317294>
- [21] W. Zhao, W. Krehling, D. Whalley, C. Healy, and F. Mueller, “Improving WCET by optimizing worst-case paths,” in *11th IEEE Real Time and Embedded Technology and Applications Symposium*, Mar. 2005, pp. 138–147, iSSN: 1545-3421. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1388381>

- [22] H. Li, I. Puaut, and E. Rohou, “Traceability of Flow Information: Reconciling Compiler Optimizations and WCET Estimation,” in *Proceedings of the 22nd International Conference on Real-Time Networks and Systems*, ser. RTNS ’14. New York, NY, USA: Association for Computing Machinery, Oct. 2014, pp. 97–106. [Online]. Available: <https://doi.org/10.1145/2659787.2659805>
- [23] —, “Tracing Flow Information for Tighter WCET Estimation: Application to Vectorization,” in *2015 IEEE 21st International Conference on Embedded and Real-Time Computing Systems and Applications*, Aug. 2015, pp. 217–226, iSSN: 2325-1301. [Online]. Available: <https://ieeexplore.ieee.org/document/7300224>
- [24] Y. Du, Z. Shen, K. Dharsee, J. Zhou, R. J. Walls, and J. Criswell, “Holistic {Control-Flow} Protection on {Real-Time} Embedded Systems with Kage,” 2022, pp. 2281–2298. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/du>
- [25] RTCA, Inc. and EUROCAE, *Software Considerations in Airborne Systems and Equipment Certification*. Washington, D.C., USA and Saint Denis Cedex, France: RTCA, Inc. and EUROCAE, 2011, jointly published as RTCA DO-178C and EUROCAE ED-12C.
- [26] International Organization for Standardization, *Road vehicles – Functional safety*. Geneva, Switzerland: ISO, 2018, consists of multiple parts.
- [27] C. Ferdinand and R. Heckmann, “ait: Worst-case execution time prediction by static program analysis,” in *Building the Information Society: IFIP 18th World Computer Congress Topical Sessions 22–27 August 2004 Toulouse, France*. Springer, 2004, pp. 377–383.
- [28] S. L. Min, R. Pettit, P. Puschner, and T. Ungerer, *Software technologies for embedded and ubiquitous systems*. Springer, 2010.
- [29] X. Li, Y. Liang, T. Mitra, and A. Roychoudhury, “Chronos: A timing analyzer for embedded software,” *Science of Computer Programming*, vol. 69, no. 1-3, pp. 56–67, 2007.
- [30] H. Falk, P. Lokuciejewski, and H. Theiling, “Design of a WCET-Aware C Compiler,” in *2006 IEEE/ACM/IFIP Workshop on Embedded Systems for Real Time Multimedia*, Oct. 2006, pp. 121–126, iSSN: 2325-1301. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4115464>
- [31] P. Lokuciejewski, S. Plazar, H. Falk, P. Marwedel, and L. Thiele, “Multi-objective Exploration of Compiler Optimizations for Real-Time Systems,” in *2010 13th IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*, May 2010, pp. 115–122, iSSN: 2375-5261. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5479565>
- [32] K. Muts and H. Falk, “Clustering Solutions of Multiobjective Function Inlining Problem,” in *21th International Workshop on Worst-Case Execution Time Analysis (WCET 2023)*, ser. Open Access Series in Informatics (OASIs), P. Wägemann, Ed., vol. 114. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, pp. 4:1–4:12, iSSN: 2190-6807. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/OASIs.WCET.2023.4>

- [33] M. Voorneveld, “Characterization of pareto dominance,” *Operations Research Letters*, vol. 31, no. 1, pp. 7–11, 2003.
- [34] A. V. Lotov and K. Miettinen, “Visualizing the pareto frontier,” in *Multiobjective optimization: interactive and evolutionary approaches*. Springer, 2008, pp. 213–243.
- [35] E. Zitzler, M. Laumanns, and L. Thiele, “Spea2: Improving the strength pareto evolutionary algorithm,” *Evolutionary Methods for Design, Optimization and Control*, vol. 202, 2001.

A Mathematical Foundations of Pareto Optimality

In the context of multi-objective optimization, where we seek to simultaneously optimize multiple conflicting objectives, the concept of Pareto optimality provides a way for understanding the trade-offs between these objectives. Consider a minimization problem with n objectives, where we have a set of feasible solutions X . Each solution $\mathbf{x} \in X$ is mapped to an objective vector $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x}))$ in the objective space.

Pareto Dominance

The notion of Pareto dominance is central to defining Pareto optimality. A solution $\mathbf{x}_1$ is said to **Pareto dominate** another solution $\mathbf{x}_2$ if $\mathbf{x}_1$ is at least as good as $\mathbf{x}_2$ in all objectives and strictly better in at least one objective. Mathematically, for a minimization problem, $\mathbf{x}_1$ Pareto dominates $\mathbf{x}_2$ ($\mathbf{x}_1 \preceq \mathbf{x}_2$) if and only if:

- (i) $f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2)$ for all objectives $i = 1, \dots, n$.
- (ii) $f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2)$ for at least one objective $j \in \{1, \dots, n\}$.

[33]

If condition (i) is met but condition (ii) is not (i.e., $f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2)$ for all i), it is sometimes referred to as weak dominance. When neither $\mathbf{x}_1$ dominates $\mathbf{x}_2$ nor $\mathbf{x}_2$ dominates $\mathbf{x}_1$, the two solutions are considered non-dominated with respect to each other.

Pareto Front

The **Pareto front** is the set of all objective vectors corresponding to Pareto optimal solutions. A solution $\mathbf{x}^* \in X$ is deemed **Pareto optimal** (or non-dominated) if no other feasible solution $\mathbf{x}' \in X$ can Pareto dominate it [34]. In other words, a solution is Pareto optimal if you cannot improve any of its objective values without worsening at least one other objective value.

The Pareto front, often denoted as PF , can be formally defined as the set of objective vectors $\mathbf{f}(\mathbf{x}^*)$ such that $\mathbf{x}^*$ is a Pareto optimal solution:

$$PF = \{\mathbf{f}(\mathbf{x}^*) \mid \mathbf{x}^* \in X \text{ and } \nexists \mathbf{x}' \in X : \mathbf{x}' \preceq \mathbf{x}^*\}$$

The set of the corresponding decision vectors $\mathbf{x}^*$ in the solution space is known as the **Pareto set**.

B The Strength Pareto Evolutionary Algorithm 2 (SPEA2)

The Strength Pareto Evolutionary Algorithm 2 (SPEA2) [35] is a widely used elitist evolutionary algorithm for multi-objective optimization. It improves upon its predecessor, SPEA, by incorporating a more precise fitness assignment strategy, a nearest-neighbor density estimation technique, and an enhanced archive truncation method. SPEA2 maintains an external archive of non-dominated solutions found throughout the evolutionary process.

The main steps of the SPEA2 algorithm are outlined in Algorithm 1.

Algorithm 1 SPEA2 Algorithm

```

1: procedure SPEA2( $N, \bar{N}, T$ )  $\triangleright N$ : Population size,  $\bar{N}$ : Archive size,  $T$ : Number of
   generations
2:   Initialize population  $P_0$  with size  $N$ .
3:   Initialize empty archive  $\bar{P}_0$ .
4:   for  $t = 0$  to  $T - 1$  do
5:     Combine population and archive:  $R_t = P_t \cup \bar{P}_t$ .
6:     Evaluate fitness of individuals in  $R_t$ .
7:     Calculate Strength Value  $S(i)$  for each individual  $i \in R_t$ :  $S(i)$  is the number
       of individuals in  $R_t$  that are dominated by or equal to  $i$ .
8:     Calculate Raw Fitness  $R(i)$  for each individual  $i \in R_t$ :  $R(i) =$ 
        $\sum_{j \in R_t, j \text{ is dominated by or equal to } i} S(j)$ .
9:     Calculate Density Information  $D(i)$  for each individual  $i \in R_t$ :  $D(i)$  is based
       on the distance to the  $k$ -th nearest neighbor in objective space. A common choice for
        $k$  is  $\sqrt{|R_t|}$ .
10:    Calculate total Fitness  $F(i)$  for each individual  $i \in R_t$ :  $F(i) = R(i) + D(i)$ .  $\triangleright$ 
       Fitness is minimized
11:    Select individuals with  $F(i) < 1$  from  $R_t$  to form the new archive  $\bar{P}_{t+1}$ .
12:    if  $|\bar{P}_{t+1}| < \bar{N}$  then
13:      Fill  $\bar{P}_{t+1}$  with the best  $|\bar{N} - |\bar{P}_{t+1}||$  individuals from  $R_t \setminus \bar{P}_{t+1}$  (those with
        $F(i) \geq 1$ ), sorted by fitness.
14:    else if  $|\bar{P}_{t+1}| > \bar{N}$  then
15:      Truncate  $\bar{P}_{t+1}$  to size  $\bar{N}$  by iteratively removing individuals that have the
       smallest distance to another individual (maintaining diversity).
16:    end if
17:    Select individuals from  $\bar{P}_{t+1}$  to form the mating pool for generating the next
       population  $P_{t+1}$ . Selection is typically done by binary tournament selection based on
       fitness.
18:    Apply genetic operators (crossover and mutation) to the mating pool to create
       the new population  $P_{t+1}$  of size  $N$ .
19:  end for
20:  Return  $\bar{P}_T$ 
21: end procedure

```

C Clustering solutions

This appendix provides the details of the clustering algorithms discussed in [32], specifically designed to handle size constraints for clustering solutions of multi-objective optimization problems.

Algorithm 2 Clustering

Input: Set $\mathcal{S} \subset \mathbb{R}^m$ to be clustered, clustering algorithm Cluster, maximum cluster size τ , maximum distance $dist_{max}$ between two clusters to be merged

Output: Set of clusters $\mathcal{S} = \{S_i\}$ with $S_i \subset \mathcal{S}$, $\bigcup S_i = \mathcal{S}$, and $S_i \cap S_j = \emptyset$ for $i \neq j$

- 1: $n \leftarrow \lceil \frac{|\mathcal{S}|}{\tau} \rceil$ ▷ Number of clusters
- 2: $\mathcal{S} \leftarrow \text{Cluster}(\mathcal{S}, n)$
- 3: $\mathcal{S} \leftarrow \text{Refine Clusters}(\mathcal{S}, \text{Cluster}, \tau)$ ▷ Algorithm 3
- 4: $\mathcal{S} \leftarrow \text{Merge Clusters}(\mathcal{S}, \tau, dist_{max})$ ▷ Algorithm 4
- 5: **return** $\mathcal{S}$

Algorithm 3 Refine clusters

Input: Set of clusters $\mathcal{S} = \{S_i\}$ with $S_i \subset \mathbb{R}^m$, clustering algorithm Cluster, maximum cluster size τ

Output: Set of refined clusters $\mathcal{R} = \{R_i\}$ with $R_i \subset \mathbb{R}^m$, $|R_i| \leq \tau$, and $\bigcup R_i = \bigcup S_i$

- 1: $C_{largest} \leftarrow \text{LargestCluster}(\mathcal{S})$
- 2: $\mathcal{R} \leftarrow \mathcal{S}$
- 3: **while** $|C_{largest}| > \tau$ **do**
- 4: $C_{refined} \leftarrow \text{Cluster}(C_{largest}, \lceil \frac{|C_{largest}|}{\tau} \rceil)$
- 5: $\mathcal{R} \leftarrow \mathcal{R} \setminus \{C_{largest}\} \cup C_{refined}$ ▷ Update $\mathcal{R}$
- 6: $C_{largest} \leftarrow \text{LargestCluster}(\mathcal{R})$
- 7: **end while**
- 8: **return** $\mathcal{R}$

Lemma 1 (from [32]). *If the input set $\mathcal{S}$ and its clusters $S_i \in \mathcal{S}$ are finite, and $0 < \tau < \infty$, Algorithm 3 terminates and returns clusters of size less than or equal to τ .*

Proof of Lemma 1. If $|S_i| \leq \tau$ for all $S_i \in \mathcal{S}$, the algorithm terminates and returns the original clusters S_i . To prove that the algorithm terminates if there exist clusters with sizes greater than τ , we prove that the while loop at Lines 3-7 terminates. We denote by $\mathcal{W}$ a set of all clusters of size greater than τ in $\mathcal{S}$:

$$\mathcal{W} = \{W \in \mathcal{S} \mid |W| > \tau\}$$

Since the input set $\mathcal{S}$ is finite, the set $\mathcal{W}$ is finite.

At the first iteration of the while loop, $C_{largest} \in \mathcal{W}$ is split into $n = \lceil \frac{|C_{largest}|}{\tau} \rceil < \infty$ clusters at Line 4. We prove by contradiction that the size of at least one of the newly created clusters is less than or equal to τ : if $|C_k^{new}| > \tau$ for all newly created clusters

C_k^{new} with $k = 1, 2, \dots, n$, then

$$|C_{largest}| = \sum_{k=1}^n |C_k^{new}| > \sum_{k=1}^n \tau = n \cdot \tau = \lceil \frac{|C_{largest}|}{\tau} \rceil \cdot \tau \geq |C_{largest}|.$$

The newly created clusters with sizes less than or equal to τ are removed from $\mathcal{W}$: $\mathcal{W} = \mathcal{W} \setminus \{C_k^{new} \mid |C_k^{new}| \leq \tau\}$. At the next iteration, the largest cluster from the updated set $\mathcal{W}$ is split into smaller clusters, and clusters with sizes less than or equal to τ are removed from $\mathcal{W}$. Since the set $\mathcal{W}$ is finite and monotonically decreases in cardinality, we continue until the set $\mathcal{W}$ is empty. It proves that the algorithm terminates.

The set $\mathcal{R}$ is updated at each iteration and

$$\mathcal{R} = \{W \in \mathcal{S} \mid |W| \leq \tau\} \cup \{W \in \mathcal{S} \mid |W| > \tau\} = \{W \in \mathcal{S} \mid |W| \leq \tau\} \cup \mathcal{W}$$

Since the algorithm terminates when $\mathcal{W} = \emptyset$, the final set $\mathcal{R}$ contains clusters of size less than or equal to τ . ■

Algorithm 4 Merge clusters

Input: Maximum cluster size τ , set of clusters $\mathcal{S} = \{S_i\}$ with $S_i \subset \mathbb{R}^m$ and $|S_i| < \tau$, maximum distance $dist_{max}$ between two clusters to be merged

Output: Set of clusters $\mathcal{R} = \{R_i\}$ with $R_i \subset \mathbb{R}^m$, $|R_i| \leq \tau$, and $\bigcup R_i = \bigcup S_i$

```

1:  $\mathcal{R} \leftarrow \mathcal{S}$ 
2: for  $C \in \mathcal{R}$  do
3:    $C_{closest} \leftarrow \text{Closest Cluster}(C, \mathcal{R})$   $\triangleright C_{closest} \neq C$ 
4:   if  $dist(C, C_{closest}) < dist_{max}$  AND  $|C| + |C_{closest}| \leq \tau$  then
5:      $C_{merged} \leftarrow \text{Merge}(C, C_{closest})$ 
6:      $\mathcal{R} \leftarrow \mathcal{R} \setminus \{C, C_{closest}\} \cup \{C_{merged}\}$   $\triangleright$  Update  $\mathcal{R}$ 
7:   go to 2
8: end if
9: end for
10: return  $\mathcal{R}$ 

```

Lemma 2 (from [32]). *If $0 < \tau < \infty$, the input set $\mathcal{S}$ is finite, and $|S_i| < \tau$ for all $S_i \in \mathcal{S}$, Algorithm 4 terminates and returns clusters of size less than or equal to τ .*

Proof of Lemma 2. To prove that the algorithm terminates, we prove that the for loop terminates. The algorithm starts with $\mathcal{S}$ assigned to $\mathcal{R}$. Since $\mathcal{S}$ is finite, $\mathcal{R}$ is finite and $|R| < \tau$ for all $R \in \mathcal{R}$. **Case 1.** If for all $C \in \mathcal{R}$, the closest cluster $C_{closest}$ does not satisfy the conditions at Line 4, the loop terminates after $|\mathcal{R}| < \infty$ iterations.

Case 2. If there exists $C \in \mathcal{R}$ such that its closest cluster $C_{closest}$ satisfies the conditions at Line 4, the clusters C and $C_{closest}$ are merged, the size of the merged cluster is less than or equal to τ , and the set $\mathcal{R}$ is updated by substituting the clusters C and $C_{closest}$ by one merged cluster. We denote by $\mathcal{R}_1$ the updated set $\mathcal{R}$. The new set $\mathcal{R}_1$ satisfies the following conditions:

- $|\mathcal{R}_1| = |\mathcal{R}| - 1 < \infty$

- $|R| \leq \tau$ for all $R \in \mathcal{R}_1$

The set $\mathcal{R}_1$ satisfies either Case 1 or Case 2 with $\mathcal{R} = \mathcal{R}_1$. Case 1 terminates the algorithm and Case 2 generates a new set of clusters which we denote by $\mathcal{R}_2$. By repeating the procedure, we generate a sequence of finite sets $\{\mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_s\}$. The sets monotonically decrease in cardinality and $|R| \leq \tau$ for all $R \in \mathcal{R}_i$, $i = 1, 2, \dots, s$, which proves the lemma. ■